Phase 2 — Sensor, model, deployment and MLOps evidence

This report is a run checklist and evidence template, not a record of experiments that have not yet been performed. All values in brackets must come from the student's own simulator/runtime output.

Sensor simulator and preprocessing

`data_pipeline/simulator.py` supports `--anomaly none`, `temp_drift`, `vibration`, and `combined`; it publishes temperature at 1 Hz, vibration RMS at 0.5 Hz, and discrete door transitions to the local Mosquitto topic tree. `data_pipeline/preprocessing.py` applies a five-value moving average to the temperature and vibration arrays and extracts a 30-second feature window every 10 seconds. The six-element vector, in fixed order, is temperature mean, standard deviation, rate °C/min, vibration RMS, peak, and Pearson kurtosis. The clean mean and standard deviation are calculated once from ten minutes of Normal simulator output, saved in `training_stats.npy`, and reused at inference. The six sensor features are fused at feature level; raw-level fusion would require clock-aligning large 500 Hz streams, while decision-level fusion would require combining independent models and would lose direct joint feature interactions. Door events are logged independently and are not an input feature in this required six-value model.

The problem has an internal label conflict. A +0.08°C-per-second drift crosses the Critical >3°C boundary after 38 seconds, although the training table calls the entire 15-minute `temp_drift` run Warning. With the chosen threshold interpretation, the dataset generator labels each drift window by the operational severity; vibration-only is an early Warning anomaly and combined drift/vibration is Critical. This change is documented in `generation_manifest.json` and should be confirmed with the instructor if the table's mode label is instead mandatory. Measured normalisation experiment: correct frozen stats accuracy **[insert]**; mean shifted by 3σ accuracy **[insert]**; change **[insert percentage points]**.

Training, TFLite and Docker

`training/train_model.py` trains a two-hidden-layer 32/16 ReLU MLP, uses a stratified 20% validation split, and exits if measured accuracy is below 88%. Record total windows, per-class counts, validation confusion matrix and Class 2 recall: **[insert actual output]**. `convert_ptq.py` requires at least 200 representative samples and exports full integer TFLite. `prune_quantise.py` applies the TFMOT PolynomialDecay schedule to whole hidden units, compacts the surviving dimensions, fine-tunes, and then converts with full INT8 calibration. Both optimization and accuracy must be checked.

The inference image uses Python 3.11 slim. Pip installation precedes the separate `COPY model.tflite .` instruction, so replacing only model bytes does not invalidate dependency layers. Record actual base image/layer bytes, rebuilt layers and registry transfer for the demo: **[insert measurements]**. Based on the supplied 280 KB estimate, a model-only update to 85 trucks is nominally 23.8 MB / ₹2.38; actual compressed layer size must be measured.

PSI and deployment

`monitoring/build_reference.py` runs the model on 300 independently simulated Normal windows, bins confidence into [0,.25), [.25,.5), [.5,.75), [.75,1], and writes `reference_dist.json`. The monitor checks its rolling 100 inference scores every 60 seconds and reports an alert over PSI 0.25. For the demo, record baseline PSI **[ ]**, injected combined PSI/time **[ ]**, and clean recovery PSI/time **[ ]**. The requested crossing within five minutes and recovery below 0.10 are targets to demonstrate, not pre-filled results; if they do not occur, preserve the logs and analyze why.

The Ansible playbook contains seven tasks: create the deployment directory; copy model; copy reference; stop only when copied artifacts change; pull the image; start with environment variables; wait 15 seconds and verify running state. Run it twice without altering inputs; attach both actual PLAY RECAPs and confirm the second reports `changed=0`. Record one genuine technical difficulty and one architecture improvement in your own words: **[student reflection required]**.

OTA decision

At decimal 280 KB/model, full replacement for 85 trucks transfers 23.8 MB and costs ₹2.38. Canary sends 2.8 MB/₹0.28 to ten trucks first and 21 MB/₹2.10 after approval; completed rollout totals the same ₹2.38. Shadow mode also distributes the candidate to trucks and adds unquantified inference/telemetry cost. A staged canary is recommended to bound blast radius for safety-sensitive cargo and pause during rural outages. Downloads must resume and validate signatures locally; a canary is safer, not cheaper after full fleet completion.